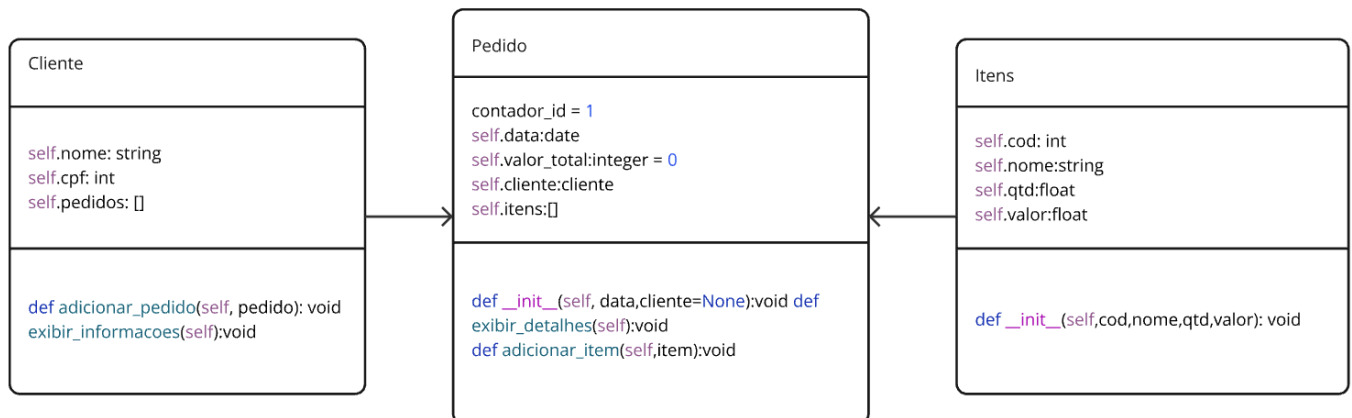


17/04 - Exemplo Classes Associação

ASSOCIAÇÃO



Exemplo de Associação:

- Cliente ---- Pedido
- Pedido ---- Itens

Explicação do Código:

- **Classe Cliente**
 - O construtor `__init__` inicializa o nome e o cpf do cliente.
 - `self.pedidos = []`: Um atributo pedidos é criado como uma lista vazia. Esta lista será usada para armazenar os objetos Pedido associados a este cliente.
 - `adicionar_pedido(self, pedido)`: Este método permite associar um objeto Pedido a este Cliente. O pedido é adicionado à lista `self.pedidos`. O código também inclui uma verificação para garantir a associação bidirecional, atualizando a referência do cliente no objeto Pedido se necessário.
 - `exibir_informacoes(self)`: Exibe as informações do cliente e lista os IDs, datas e valores totais dos pedidos associados.
- **Classe Pedido**
 - `contador_id = 1`: Uma variável de classe para gerar IDs únicos para cada pedido.

- O construtor `__init__` inicializa o id (usando o contador), a data, o valor_total e, opcionalmente, um cliente.
- `self.cliente = cliente`: Um atributo cliente é criado para armazenar uma referência ao objeto Cliente que fez este pedido.
- Dentro do construtor de Pedido, se um cliente for fornecido, o método `adicionar_pedido` do cliente é chamado para estabelecer a associação bidirecional.
- `exibir_detalhes(self)`: Exibe os detalhes do pedido, incluindo o ID, a data, o valor total e o nome do cliente associado (se houver).
- `adicionar_item(self, item)`: adiciona um objeto Itens na lista de pedidos.
- **Classe Itens**
 - O construtor `__init__` inicializa os atributos do Objeto Ítem

Conceito de Associação:

A associação é uma relação entre duas classes onde os objetos dessas classes estão relacionados de alguma forma, mas têm ciclos de vida independentes. Um objeto de uma classe pode usar ou conhecer objetos de outra classe.

No exemplo acima:

- Um Cliente pode ter vários Pedidos associados (relação um-para-muitos).
- Um Pedido pode estar associado a um Cliente (relação muitos-para-um do ponto de vista do pedido, ou um-para-um se considerarmos o pedido específico).

A associação é implementada através da manutenção de referências entre os objetos. No Cliente, temos uma lista de Pedidos, e no Pedido, temos uma referência ao Cliente que o realizou.

Associação Bidirecional (Opcional):

O código também demonstra como criar uma associação bidirecional. Isso significa que tanto o Cliente conhece seus Pedidos quanto o Pedido conhece o seu Cliente. Isso pode ser útil em certas situações para navegar entre os objetos de forma mais fácil. A bidirecionalidade é estabelecida nas seguintes partes:

- No método `adicionar_pedido` da classe Cliente, onde se verifica e atualiza a referência cliente no objeto Pedido.
- No construtor da classe Pedido, onde se chama `cliente.adicionar_pedido(self)` se um cliente for fornecido.
- No exemplo de uso, ao associar `pedido4` a `cliente2` posteriormente, ambas as referências são atualizadas.

A escolha de implementar uma associação unidirecional ou bidirecional depende dos requisitos específicos do seu programa.

```

class Cliente:
    def __init__(self, nome, cpf):
        self.nome = nome
        self.cpf = cpf
        self.pedidos = [] # Lista para armazenar os pedidos associados a este cliente

    def adicionar_pedido(self, pedido):
        self.pedidos.append(pedido)
        # Estabelecendo a associação bidirecional (se desejado)
        if pedido.cliente != self:
            pedido.cliente = self

    def exibir_informacoes(self):
        print(f"Cliente: {self.nome} (CPF: {self.cpf})")
        if self.pedidos:
            print("Pedidos:")
            for pedido in self.pedidos:
                print(f"- ID: {pedido.id}, Data: {pedido.data}, Valor Total: R$
{pedido.valor_total:.2f}")
        else:
            print("Nenhum pedido realizado.")

class Itens:
    def __init__(self, cod, nome, qtd, valor):
        self.cod = cod
        self.nome = nome
        self.qtd = qtd
        self.valor = valor

class Pedido:
    contador_id = 1

    def __init__(self, data, cliente=None):
        self.id = Pedido.contador_id
        Pedido.contador_id += 1
        self.data = data
        self.valor_total = 0
        self.cliente = cliente # Referência ao cliente que fez o pedido
        self.itens = [] # lista de itens
        # Estabelecendo a associação bidirecional (se o cliente for fornecido)
        if cliente:
            cliente.adicionar_pedido(self)

    def exibir_detalhes(self):
        print(f"Detalhes do Pedido ID: {self.id}")
        print(f>Data: {self.data}")

        if self.cliente:

```

```

        print(f"Cliente: {self.cliente.nome}")
    else:
        print("Cliente não associado.")
    total = 0
    for i in self.itens:
        print(f'cod {i.cod} nome {i.nome} qtd {i.qtd} valor {i.valor} total=
{i.qtd*i.valor}')
        total+=i.qtd*i.valor
    print(f"Valor Total: R$ {self.valor_total:.2f}")
    # print(f'Total Pedido {total}')
def adicionar_item(self,item):
    self.itens.append(item)
    totalItem=item.qtd * item.valor
    self.valor_total+=totalItem

# Exemplo de uso:
if __name__ == "__main__":
    # Criando clientes
    cliente1 = Cliente("Ana Souza", "123.456.789-00")
    cliente2 = Cliente("Pedro Alves", "987.654.321-11")

    # Criando pedidos e associando aos clientes
    pedido1 = Pedido("2025-04-10", cliente1)
    item1=Itens(123,'Mouse',1,39.90)
    item2=Itens(2424,'Teclado',3,50.40)
    pedido1.adicionar_item(item1)
    pedido1.adicionar_item(item2)
    pedido1.exibir_detalhes()

    pedido2 = Pedido("2025-04-05", cliente1)
    pedido3 = Pedido("2025-04-08", cliente2)
    pedido4 = Pedido("2025-04-01") # Pedido sem cliente inicial

    # Associando um pedido a um cliente posteriormente
    cliente2.adicionar_pedido(pedido4)
    # pedido4.cliente = cliente2 # Garantindo a bidirecionalidade

    print("--- Informações dos Clientes ---")
    cliente1.exibir_informacoes()
    print("\n")
    cliente2.exibir_informacoes()
    print("\n")

    print("--- Detalhes dos Pedidos ---")
    pedido1.exibir_detalhes()
    print("\n")
    pedido3.exibir_detalhes()

```

```
print("\n")  
pedido4.exibir_detalhes()
```